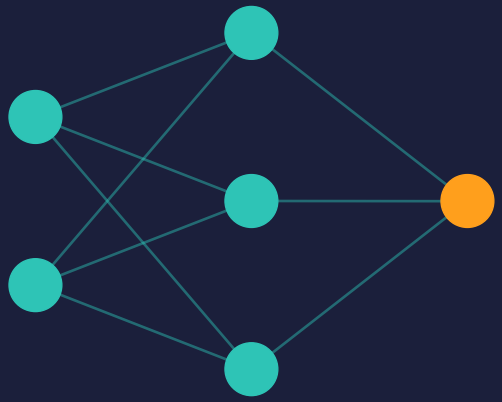
دوره یادگیری عمیق و شبکه‌های عصبی



## جلسه ۳

# مبانی PyTorch: تنسور، Autograd و GPU

از NumPy و Backpropagation دستی به چارچوب استاندارد یادگیری عمیق

کیوان محمودی

# در این جلسه

۱ چرا به یک چارچوب نیاز داریم؟

۲ تنسور: shape، dtype و device

۳ تغییر شکل، Indexing و Broadcasting

۴ ارتباط با NumPy

۵ محاسبات روی GPU

۶ مشتق‌گیری خودکار (Autograd)

۷ detach، no_grad و انباشت گرادیان

۸ اولین مدل با nn.Linear و optim

# یک چارچوب یادگیری عمیق چه می‌کند؟

PyTorch (Meta AI، ۲۰۱۶) رایج‌ترین چارچوب پژوهشی و پایه اغلب مدل‌های Hugging Face است.

### ۱ محاسبات تنسوری

عملیات ماتریسی سریع و بهینه روی CPU و GPU.

### ۲ Autograd

محاسبه خودکار گرادیان‌ها؛ نسخه صنعتی کلاس Value جلسه قبل.

### ۳ بلوک‌های آماده

لایه‌ها (nn)، توابع هزینه، بهینه‌سازها (optim) و بارگذاری داده (DataLoader).

# تنسور (Tensor): آرایه چندبعدی




scalar
rank 0
shape ()

vector
rank 1
shape (4,)

matrix
rank 2
shape (3, 4)

rank 3
shape (3, 3, 4)
e.g. (batch, seq, emb)

rank 4
(N, C, H, W)
batch of images

تصویرها در PyTorch به شکل (N, C, H, W) و متن‌ها به شکل (batch, seq_len, emb_dim) نمایش داده می‌شوند

# ساخت تنسور و ویژگی‌های اصلی آن

```
x = torch.tensor([[1., 2.], [3., 4.]])
z = torch.zeros(2, 3)
r = torch.randn(3, 4)            # standard normal
a = torch.arange(0, 10, 2)

x.shape    # torch.Size([2, 2])
x.dtype    # torch.float32  (default)
x.device   # cpu

labels = torch.tensor([0, 2, 1])
labels.dtype   # torch.int64 -> required by CrossEntropyLoss
```

- سه ویژگی را همیشه بررسی کنید: shape، dtype و device.
- نوع پیش‌فرض float32 است (در NumPy float64).
- بیشتر خطاهای رایج از ناسازگاری ابعاد یا نوع داده‌اند.

# تغییر شکل و Indexing

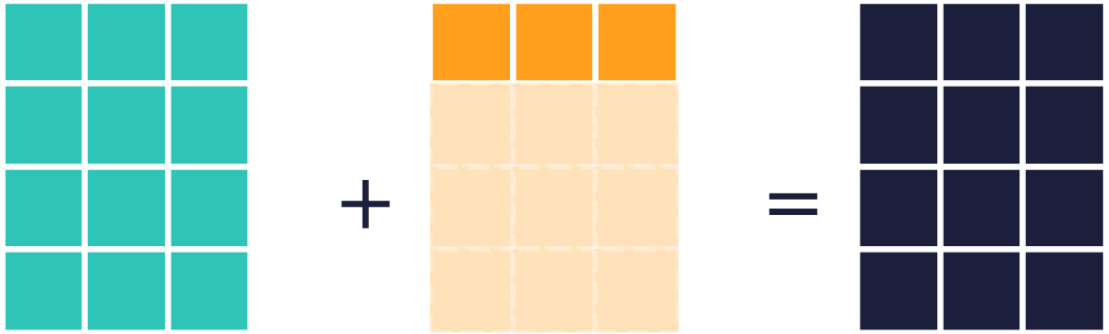
```
t = torch.arange(24)
t.view(2, 3, 4)          # same data, new shape
t.reshape(4, -1)         # -1: infer this dimension

img = torch.rand(32, 32, 3)        # H, W, C
img = img.permute(2, 0, 1)         # C, H, W
batch = img.unsqueeze(0)           # 1, C, H, W
batch.squeeze(0)                   # C, H, W

m = torch.arange(12).view(3, 4)
m[1], m[:, 2], m[m > 5]            # like NumPy
```

- view/reshape: تغییر شکل بدون تغییر داده.
- permute: جابه‌جایی ترتیب ابعاد.
- unsqueeze/squeeze: افزودن یا حذف بعد با اندازه ۱ (مثلاً بعد batch).

# Broadcasting: ترکیب تنسورهای هم‌خوان




X @ W  (4, 3)

bias (3,) -> (4, 3)

output (4, 3)

Rule: compare shapes from the right; sizes must be equal or 1

بایاس با شکل (3,) به‌طور خودکار روی همه سطرهای دسته تکرار می‌شود

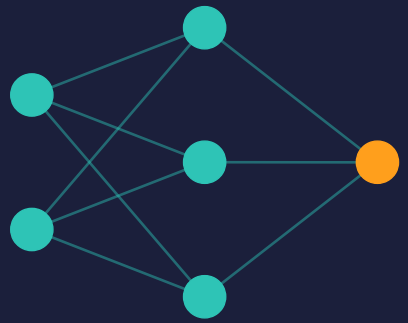
# پل NumPy و GPU

## پل NumPy

- torch.from_numpy(arr) و tensor.numpy بدون کپی‌اند.
- حافظه مشترک: تغییر یکی، دیگری را هم تغییر می‌دهد.
- برای کپی مستقل: torch.tensor(arr) یا clone.
- برای رسم نمودار: t.detach.cpu.numpy

## انتقال به GPU

- انتخاب دستگاه: cuda در صورت وجود GPU، وگرنه cpu
- x = x.to(device) و model.to(device)
- همه تنسورهای یک عملیات باید روی یک دستگاه باشند.
- Colab: Runtime → Change runtime type در → T4 GPU

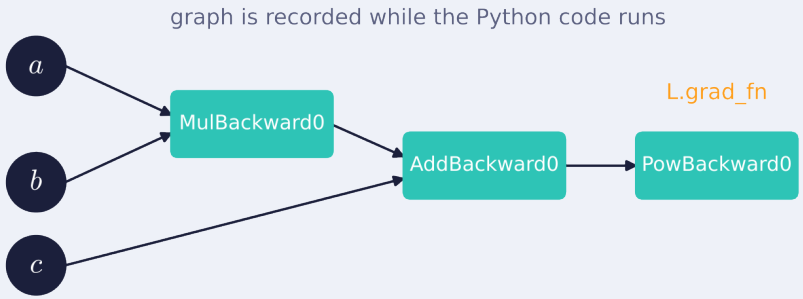


Autograd

# مشتق‌گیری خودکار

همان کاری که کلاس Value انجام می‌داد، برای تنسورها و بسیار سریع

# گراف محاسباتی پویا (Dynamic Graph)




- با requires_grad=True همه عملیات ثبت می‌شوند.
- گراف همزمان با اجرای کد پایتون ساخته می‌شود.
- هر تنسور خروجی یک grad_fn دارد.
- با فراخوانی L.backward گرادیان‌ها در a.grad و ... نوشته می‌شوند.

# Autograd در عمل

```
a = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(-3.0, requires_grad=True)
c = torch.tensor(10.0, requires_grad=True)

L = (a * b + c) ** 2
L.backward()
a.grad, b.grad, c.grad    # -24, 16, 8  (same as session 2)

with torch.no_grad():     # no graph: evaluation / manual updates
    y = a * 2

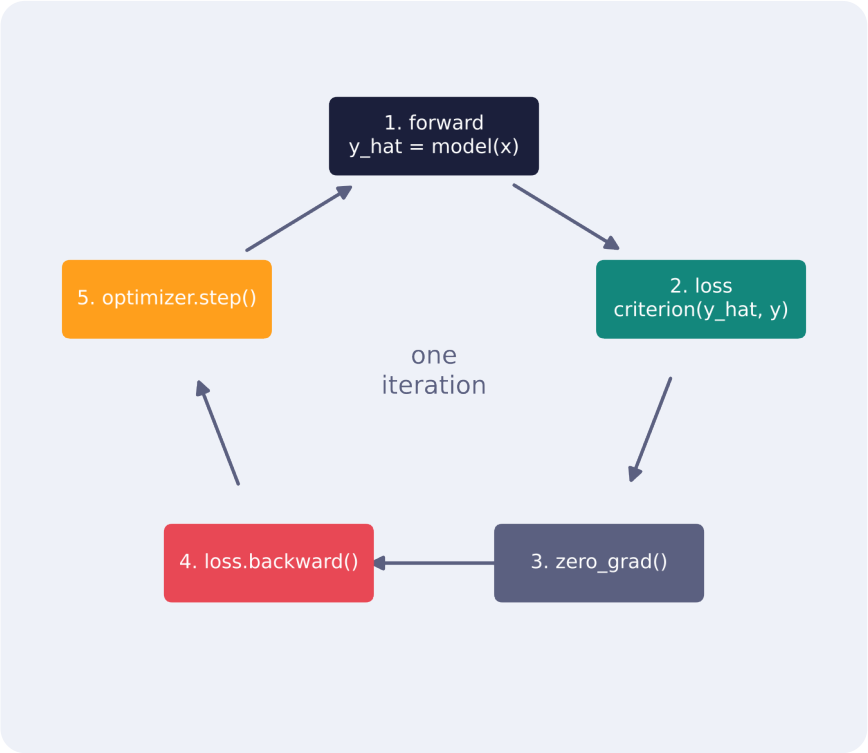
loss_value = L.item()     # Python float, detached from the graph
```

- گرادیان‌ها انباشته می‌شوند: قبل از هر backward باید صفر شوند.
- no_grad برای ارزیابی مدل و به‌روزرسانی دستی وزن‌ها.
- برای جدا کردن مقدار از گراف: item و detach

# تطبیق Backpropagation دستی با Autograd

- شبکه دولایه جلسه ۲ را با تنسورها ساختیم و loss را با cross_entropy حساب کردیم.
- گرادیان‌های autograd را با فرمول‌های دستی N/(P - Y) و … مقایسه کردیم.
- نتیجه: torch.allclose برای W2 ،W1 و b1 مقدار True داد.
- از این به بعد هیچ‌وقت گرادیان را دستی نمی‌نویسیم، اما می‌دانیم پشت صحنه چه می‌گذرد.

# حلقه آموزش استاندارد در پنج مرحله




- forward: محاسبه پیش‌بینی
- loss: مقایسه با برچسب واقعی
- zero_grad: پاک کردن گرادیان‌های قبلی
- backward: محاسبه گرادیان‌ها
- step: به‌روزرسانی پارامترها
- این الگو در همه جلسات بعدی تکرار می‌شود.

# رگرسیون خطی با nn.Linear و optim.SGD

```
model = nn.Linear(in_features=1, out_features=1)
criterion = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

for epoch in range(100):
    y_hat = model(x)                # 1. forward
    loss = criterion(y_hat, y)      # 2. loss
    optimizer.zero_grad()           # 3. reset gradients
    loss.backward()                 # 4. backpropagation
    optimizer.step()                # 5. update

# learned: w = 2.98, b = -1.98   (true: 3, -2)
```

- nn.Linear وزن و بایاس را می‌سازد و مقداردهی می‌کند.
- model.parameters همه پارامترهای قابل آموزش را برمی‌گرداند.
- مدل پارامترهای واقعی را با دقت خوبی پیدا کرد.

# جمع‌بندی

- تنسور = آرایه چندبعدی با سه ویژگی کلیدی: shape، dtype و device.
- permute، reshape، view و unsqueeze پرکاربردترین تغییر شکل‌ها هستند.
- from_numpy حافظه مشترک دارد؛ مدل و داده باید روی یک device باشند.
- Autograd با گراف پویا گرادیان‌ها را خودکار محاسبه می‌کند.
- گرادیان‌ها انباشته می‌شوند؛ no_grad برای ارزیابی.
- حلقه آموزش: forward → loss → zero_grad → backward → step

# تمرین

۱ دسته‌ای از ۱۶ تصویر ۶۴×۶۴ رنگی را از شکل (16, 64, 64, 3) به (16, 3, 64, 64) ببرید و میانگین هر کانال را حساب کنید.

۲ با autograd مشتق $\sin(x)\cdot\exp(-x^2)$ را در ۱۰۰ نقطه حساب و با مشتق تحلیلی مقایسه کنید.

۳ در رگرسیون دستی خط zero_ را حذف کنید؛ چه اتفاقی می‌افتد و چرا؟

۴ داده $y = 0.5x^3 - x$ را یک بار با nn.Linear(1, 1) و یک بار با ویژگی‌های $[x, x^2, x^3]$ مدل کنید.